



Master Test Plan - CodeSmile

Autore	Matricola
Daniele Pio Scaparra	NF22500101
Grazia Bonagura	NF22500105
Antonio Nappi	NF22500200

Indice

1. Introduzione.....	2
2. Obiettivi del Testing.....	2
3. Scope del Testing.....	2
3.1 Componenti in Scope.....	2
3.2 Componenti fuori Scope.....	3
4. Strategia di Testing.....	3
4.1 Livelli di Test.....	3
4.2 Approccio per Livello di Test.....	3
4.2.1 Unit Testing.....	3
4.2.2 Integration Testing.....	4
4.2.3 System Testing.....	4
5. Coverage.....	5
5.1 Unit Testing Coverage.....	5
5.2 Integration Testing Coverage.....	5
5.3 System Testing Coverage.....	5
6. Criteri di Accettazione.....	6

1. Introduzione

Il presente **Master Test Plan** definisce la strategia complessiva di testing adottata per il progetto *CodeSmile*.

Il documento descrive gli obiettivi, lo scope, l'approccio metodologico e i criteri di accettazione delle attività di test, fornendo una visione di alto livello del processo di verifica e validazione del sistema.

Il Master Test Plan si colloca al livello strategico della documentazione di testing e costituisce il riferimento per i piani di test di livello inferiore (unit, integration e system testing), nonché per la gestione degli incidenti e delle attività di regression testing successive alle modifiche previste.

2. Obiettivi del Testing

Gli obiettivi principali dell'attività di testing sono:

- **Stabilire una baseline affidabile** del comportamento del sistema prima dell'applicazione delle Change Request approvate, da utilizzare come riferimento per le successive attività di regression testing.
 - **Verificare la correttezza funzionale** delle componenti core di CodeSmile coinvolte dalle modifiche previste, assicurando che i requisiti funzionali siano soddisfatti.
 - **Ridurre il rischio di regressioni** attraverso il riutilizzo sistematico della suite di test esistente e l'estensione mirata dei test in funzione delle Change Request introdotte.
 - **Supportare il processo di manutenzione correttiva, perfettiva ed evolutiva** del sistema, fornendo evidenze oggettive sulla stabilità e sulla qualità del software.
 - **Documentare in modo tracciabile** i risultati delle attività di testing e gli eventuali incidenti rilevati, distinguendo tra difetti funzionali, problemi ambientali e componenti fuori scope.
-

3. Scope del Testing

3.1 Componenti in Scope

In accordo con le Change Request approvate, l'attività di testing è limitata alle componenti direttamente coinvolte o impattate dalle modifiche previste.

Le componenti **in scope** includono:

- Static Analysis CLI
- Core engine di analisi statica:
 - ProjectAnalyzer
 - Inspector
 - RuleChecker
- Detection Rules (con particolare riferimento al detector *In-Place APIs Misused*)
- Code Extractors

- Modulo di Report Generation

3.2 Componenti fuori Scope

Le seguenti componenti sono esplicitamente **fuori scope** per l'attività di testing pianificata nel presente documento:

- AI-Based Detection e pipeline di fine-tuning
- Web Application basata su microservizi
- Interfaccia grafica (GUI Tkinter)

Tali componenti non sono coinvolte dalle Change Request considerate e presentano dipendenze ambientali o infrastrutturali che esulano dagli obiettivi del presente piano di test.

4. Strategia di Testing

La strategia di testing adottata prevede l'utilizzo di diversi livelli di test, ciascuno caratterizzato da un approccio specifico e coerente con gli obiettivi di verifica.

L'attività di testing si basa sul **riutilizzo della suite di test già esistente**, sviluppata nelle fasi precedenti del progetto, che viene **estesa e adattata** in funzione delle Change Request approvate, al fine di validare le nuove funzionalità introdotte e prevenire regressioni sul comportamento esistente.

4.1 Livelli di Test

Il piano di test prevede i seguenti livelli:

- **Unit Testing**
- **Integration Testing**
- **System Testing**

Il testing end-to-end della Web Application non è previsto in quanto fuori scope.

4.2 Approccio per Livello di Test

4.2.1 Unit Testing

L'unit testing adotta un **approccio white-box**.

Obiettivi principali:

- Validare il comportamento delle singole classi e funzioni in modo isolato.
- Verificare la correttezza dei rami decisionali e delle condizioni logiche.
- Ridurre il rischio di falsi positivi e falsi negativi nelle regole di detection.

L'approccio white-box è ritenuto particolarmente adeguato in quanto le componenti core di CodeSmile implementano logiche rule-based e condizioni decisionali complesse che richiedono un'esercitazione esplicita dei rami di esecuzione.

4.2.2 Integration Testing

L'integration testing adotta un **approccio gray-box**, combinando la verifica del comportamento esterno dei componenti con una conoscenza parziale della loro struttura e delle loro responsabilità interne.

Obiettivi principali:

- Verificare la **corretta interazione tra i componenti core del sistema**, in particolare lungo il flusso di analisi che coinvolge CLI, ProjectAnalyzer, Inspector, RuleChecker e generazione dei risultati.
- Validare il **flusso funzionale integrato** dalla fase di input dell'utente fino alla produzione dell'output finale.
- Individuare eventuali **difetti di integrazione** non rilevabili a livello unitario, quali errori di coordinamento, passaggio di dati non corretto o gestione incompleta delle condizioni di errore.

L'approccio gray-box consente di progettare i test di integrazione tenendo conto delle interfacce e delle responsabilità dei moduli coinvolti, senza richiedere l'esercitazione esaustiva dei rami decisionali interni, che rimane demandata all'unit testing.

4.2.3 System Testing

Il system testing è condotto con un **approccio black-box**.

Obiettivi principali:

- Validare il comportamento osservabile del sistema nel suo insieme.
- Verificare la gestione degli input, dei parametri e delle condizioni ambientali.
- Confermare la conformità del sistema ai requisiti funzionali dichiarati.

La progettazione dei test di sistema è effettuata mediante il **Category Partition Method** e il criterio di **Weak Equivalence Class Testing**, con l'obiettivo di garantire la copertura sistematica delle classi di input valide e invalide e delle condizioni ambientali rilevanti.

Il dettaglio dei test frame, delle categorie identificate e dei casi di test derivati è documentato nel documento *Pre-Modification System Testing*, che costituisce il riferimento operativo per l'esecuzione dei test di sistema.

5. Coverage

La coverage del codice è utilizzata come metrica di supporto alla valutazione della completezza delle attività di testing, con modalità e obiettivi differenti in funzione del livello di test considerato.

La misurazione della coverage viene effettuata tramite **coverage.py**, integrato nell'esecuzione dei test mediante il plugin **pytest-cov**, con abilitazione della **branch coverage**.

5.1 Unit Testing Coverage

Per l'**unit testing**, condotto con approccio **white-box**, la coverage costituisce un **criterio di qualità rilevante**.

La baseline di riferimento, calcolata sulla suite di unit test esistente e sulle componenti core in scope, risulta pari a una **branch coverage complessiva compresa tra l'80% e l'83%**.

L'obiettivo del piano di test è **evitare regressioni di copertura**, garantendo che la branch coverage degli unit test **non scenda al di sotto di tale baseline** durante le attività di manutenzione e di regression testing successive alle Change Request.

La coverage è valutata esclusivamente sui package core coinvolti dalle modifiche previste; sono esplicitamente esclusi dalla valutazione i moduli fuori scope (es. GUI, Web Application, componenti non rilevanti per le Change Request).

5.2 Integration Testing Coverage

Per l'**integration testing**, la coverage viene raccolta **a fini osservativi e migliorativi**, al fine di valutare l'ampiezza delle porzioni di codice esercitate dai flussi integrati del sistema.

La baseline corrente di integration testing evidenzia una coverage significativamente inferiore rispetto all'unit testing, in quanto tale livello di test è finalizzato alla validazione delle interazioni tra componenti e dei flussi funzionali principali, e non all'esercitazione sistematica dei rami decisionali interni.

L'obiettivo per l'integration testing è un **miglioramento progressivo della coverage**, con un target indicativo realistico nell'ordine del **40-50%** (di conseguenza superiore al 27% a cui si attesta attualmente), compatibilmente con la natura dei test e senza introdurre casi di test artificiali finalizzati esclusivamente all'aumento della metrica.

La coverage di integration testing **non costituisce criterio di accettazione**, ma rappresenta un indicatore di supporto all'analisi della qualità complessiva dei flussi integrati.

5.3 System Testing Coverage

Per il system testing, condotto con approccio **black-box**, la coverage del codice non costituisce una metrica significativa ai fini della valutazione della qualità del testing.

In tale contesto, la validazione del sistema è basata principalmente sulla copertura dello spazio degli input e dei comportamenti osservabili, piuttosto che sull'esercitazione dei rami interni del codice.

La progettazione dei test di sistema è effettuata mediante il **Category Partition Method** e il criterio di **Weak Equivalence Class Testing**, con l'obiettivo di garantire la copertura sistematica delle classi di input valide e invalide e delle condizioni ambientali rilevanti.

Eventuali informazioni di coverage raccolte durante l'esecuzione dei test di sistema hanno carattere puramente informativo e non costituiscono criterio di accettazione.

6. Criteri di Accettazione

Il sistema è considerato **accettabile** al termine delle attività di testing se **tutte** le seguenti condizioni risultano soddisfatte:

1. **Esecuzione completa dei test pianificati**
Tutti i test definiti per le componenti dichiarate *in scope* nel presente Master Test Plan sono stati eseguiti almeno una volta, secondo le modalità e i livelli di test previsti (unit, integration e system testing).
2. **Assenza di difetti funzionali critici aperti**
Non risultano presenti difetti funzionali classificati come *critici* o *bloccanti* al termine delle attività di testing. Eventuali difetti non critici devono essere documentati e valutati come accettabili rispetto allo scope del progetto.
3. **Coerenza del comportamento osservabile**
Il comportamento del sistema osservato durante il system testing è coerente con le specifiche funzionali definite, inclusa la corretta gestione degli input validi e invalidi e delle principali condizioni di errore.
4. **Assenza di regressioni funzionali**
Le modifiche introdotte dalle Change Request non devono introdurre regressioni rispetto al comportamento della baseline pre-modifica, come verificato tramite l'esecuzione dei test di regression testing.
5. **Mantenimento della coverage di unit testing**
La branch coverage degli unit test, misurata tramite *coverage.py* integrato in *pytest*, non deve scendere al di sotto della baseline di riferimento stabilita (80–83%) per le componenti core in scope.
6. **Gestione e tracciabilità degli incidenti di test**
Tutti gli incidenti rilevati durante le attività di testing sono stati correttamente documentati, classificati e valutati in termini di severità e impatto, distinguendo tra difetti funzionali, problemi ambientali e componenti fuori scope.